

CASE STUDY 3

Sensor Monitoring using Segment Tree

Smart Warehouse IoT Temperature Monitoring System

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025-26) : { 1st year, 3rd term }
Faculty Name	Dr. Jayaram Boga
Student Name	P. Bindhu Madhavi
Roll Number	2520030077
Branch	CSE

Course Outcome CO3: Apply Segment Tree data structure to construct range query systems, demonstrating $O(\log n)$ range maximum queries and point updates for dynamic datasets in real-world IoT sensor monitoring scenarios.

SCENARIO

A smart warehouse uses IoT sensors to monitor temperature across machines. Each sensor continuously records readings, and the system must efficiently handle queries to find the maximum temperature within a given range. It must also support updates when sensor readings change.

To ensure fast performance, a **Segment Tree** data structure is used, enabling efficient range queries and updates in $O(\log n)$ time.

Sensor Data Array

Index	0	1	2	3	4	5	6	7
Value (°C)	45	50	48	60	55	62	58	65

WORK DONE SUMMARY

Part (a) Segment Tree Construction & Range Query

Built a full segment tree for 8 sensor readings. Leaf nodes store original values; internal nodes store the maximum of their children. Demonstrated the range maximum query for indices 2–6, visiting only relevant nodes in $O(\log n)$ time.

Part (b) Range Maximum Query Algorithm

Developed a recursive algorithm handling three cases: No Overlap (prune subtree, return -INF), Complete Overlap (return stored node value), and Partial Overlap (recurse on both children). Traced the query [2,6] step-by-step, yielding max = 62.

Part (c) Segment Tree vs Fenwick Tree vs Sparse Table

Compared all three data structures across query type, update support, time/space complexity, and suitability for dynamic sensor data. Justified why Segment Tree is optimal for real-time IoT monitoring due to $O(\log n)$ updates and flexible aggregate support.

IMPLEMENTATION DETAILS

Class Design, Construction & Query

```
class SegmentTree:
    def __init__(self, data):
        self.n = len(data)
        self.tree = [0]*(2*self.n) # size 2n
        for i,v in enumerate(data):
            self.tree[self.n+i] = v # load leaves
        for i in range(self.n-1,0,-1): # build
            self.tree[i] = max(self.tree[2*i],
                              self.tree[2*i+1])
        def query(self, L, R): # range max
            res = float('-inf')
            L += self.n; R += self.n+1 # to leaf layer
            while L < R:
                if L&1: res=max(res,self.tree[L]); L+=1
                if R&1: R-=1; res=max(res,self.tree[R])
                L>>=1; R>>=1
            return res
        def update(self, i, val): # point update
            p = i+self.n
            self.tree[p] = val
            p >>= 1
            while p>=1:
                self.tree[p]=max(self.tree[2*p],self.tree[2*p+1])
                p>>=1
```

Array Layout
Leaves: tree[n..2n-1]
Internal: tree[1..n-1]
Root: tree[1]

build() — O(n)
Load leaves, then fill internal nodes bottom-up as max of children.

query(L,R) — O(log n)
Iterative; accumulate max from boundary nodes. Half-open [L,R).

update(i,val) — O(log n)
Set leaf, then recompute each ancestor up to root.

Demo & Expected Output

```
sensors = [45,50,48,60,55,62,58,65]
st = SegmentTree(sensors)
print(st.tree[1]) # 65 - global max
print(st.query(2,6)) # 62 - range max
st.update(5, 70) # sensor 5 → 70°C
print(st.query(2,6)) # 70 - updated
print(st.query(0,3)) # 60
print(st.query(4,7)) # 70
```

Output
65
62
70
60
70

Complexity Summary

Operation	Time	Space	Notes
Build	O(n)	O(n)	Bottom-up pass; 2n array entries
Range Max Query	O(log n)	O(1)	Boundary nodes only; subtrees pruned
Point Update	O(log n)	O(1)	Path from leaf to root (log n steps)

QUESTIONS

- (a)** Construct a segment tree for the given sensor readings. Explain how values are calculated at each level and how maximum values propagate from leaf nodes to the root. Demonstrate a range maximum query on the constructed tree.
- (b)** Develop an algorithm for range maximum queries using a segment tree. Handle no overlap, partial overlap, and complete overlap cases. Explain how the recursive approach reduces nodes visited and improves efficiency.
- (c)** Compare the segment tree with Fenwick Tree and Sparse Table for range queries. Discuss advantages and limitations in terms of updates and query performance. Justify why segment tree is most suitable for dynamic sensor monitoring datasets.

ANSWERS

(a) Construction of Segment Tree

A Segment Tree is a binary tree where leaf nodes store original sensor readings and internal nodes store the maximum value of their child subtree.

Step 1: Leaf Nodes

Index	0	1	2	3	4	5	6	7
Value	45	50	48	60	55	62	58	65

Step 2: Build Internal Nodes (Level by Level)

Level	Pairs	Max Values	Result
Level 1	(45,50), (48,60), (55,62), (58,65)	50, 60, 62, 65	[50, 60, 62, 65]
Level 2	(50,60), (62,65)	60, 65	[60, 65]
Root	(60,65)	65	Root = 65

Segment Tree Structure

65	← Root
60 65	
50 60 62 65	
45 50 48 60 55 62 58 65	← Leaf nodes
[0][1][2][3][4][5][6][7]	← Indices

Range Maximum Query Demo — Indices 2 to 6

Query: Find maximum temperature from sensor index 2 to 6 → subarray [48, 60, 55, 62, 58]

Node Range	Action	Value Used
[0,1]	No overlap — skip	—
[2,3]	Complete overlap — use directly	60
[4,5]	Complete overlap — use directly	62
[6,6]	Complete overlap — use directly	58
Result	max(60, 62, 58)	62 °C

(b) Algorithm for Range Maximum Query

RangeMax(node, start, end, L, R):
1. IF end < L OR start > R: // No Overlap
return -INF // prune this branch
2. IF L <= start AND end <= R: // Complete Overlap
return tree[node] // use stored value directly
3. ELSE: // Partial Overlap
mid = (start + end) / 2
leftMax = RangeMax(leftChild, start, mid, L, R)
rightMax = RangeMax(rightChild, mid+1, end, L, R)
return max(leftMax, rightMax)

No Overlap	Node range [0,1], Query [4,6]. No common elements. Return -INF → branch pruned, no further recursion.
Complete Overlap	Node range [4,5], Query [2,6]. $[4,5] \subseteq [2,6]$. Return stored value directly; children not traversed.
Partial Overlap	Node range [0,3], Query [2,6]. Elements 2–3 overlap, 0–1 do not. Recurse on both children; return max.

Example Traversal — Query max in range [2, 6]

Step	Node Range	Case	Action / Value
1	[0,7]	Partial	Recurse left [0,3] and right [4,7]
2	[0,3]	Partial	Recurse left [0,1] and right [2,3]
3	[0,1]	No Overlap	Return -INF — subtree PRUNED
4	[2,3]	Complete	Return stored value: 60
5	[4,7]	Partial	Recurse left [4,5] and right [6,7]
6	[4,5]	Complete	Return stored value: 62
7	[6,7]	Partial	Recurse left [6,6] and right [7,7]
8	[6,6]	Complete	Return stored value: 58
9	[7,7]	No Overlap	Return -INF — subtree PRUNED
Result	[2,6]	—	$\max(60, 62, 58) = 62\text{ }^{\circ}\text{C}$

(c) Comparison: Segment Tree vs Fenwick Tree vs Sparse Table

Feature	Segment Tree	Fenwick Tree	Sparse Table
Query Type	Range queries	Prefix/range sum	Static range queries
Supports Max Query	Yes	Limited	Yes
Update Support	Excellent	Good	Poor
Range Query Time	$O(\log n)$	$O(\log n)$	$O(1)$
Update Time	$O(\log n)$	$O(\log n)$	$O(n)$
Suitable for Dynamic	Yes	Partially	No
Memory Usage	Higher	Low	Moderate
Implementation	Moderate	Simple	Simple

Why Segment Tree is Best for Sensor Monitoring

Sensor systems are dynamic — temperatures change continuously, requiring frequent updates and real-time range queries.

- **Updates — $O(\log n)$:** Only the path from the changed leaf to the root is updated; no full rebuild.
- **Queries — $O(\log n)$:** Irrelevant subtrees are pruned; useful nodes are returned directly.
- **vs Sparse Table:** Avoids the $O(n)$ rebuild cost that disqualifies Sparse Table for dynamic use.
- **vs Fenwick Tree:** More flexible — handles max, min, sum, and other aggregates, not just prefix sums.

RESULT OUTPUT SUMMARY

Segment Tree — Final Structure

Level	Nodes (Max values)
Root	65
Level 2	60 65
Level 1	50 60 62 65
Leaf (Level 0)	45 50 48 60 55 62 58 65

Part (a) — Segment Tree Construction

Tree built in $O(n)$ time. Root = 65 (global max).
Range query [2,6] visits only 4 relevant nodes → answer = 62 °C.

Part (b) — Range Query Result

Query $\text{RangeMax}([2,6])$ traverses 9 nodes total, prunes 2 (No Overlap).
Nodes used: [2,3]=60, [4,5]=62, [6,6]=58 → max = 62 °C. Time: $O(\log n)$.

Part (c) — Best Structure for Dynamic Sensor Data

Segment Tree is optimal: $O(\log n)$ updates + $O(\log n)$ queries + supports max/min/sum.
Sparse Table: fast $O(1)$ queries but $O(n)$ rebuild cost → unsuitable for dynamic data.

Complexity Summary

Operation	Segment Tree	Fenwick Tree	Sparse Table
Build Time	$O(n)$	$O(n \log n)$	$O(n \log n)$
Range Max Query	$O(\log n)$	$O(\log n)^*$	$O(1)$
Point Update	$O(\log n)$	$O(\log n)$	$O(n)$ rebuild
Space	$O(n)$	$O(n)$	$O(n \log n)$
Dynamic Updates	Yes	Partial	No